

Shortest Path Routing Optimization Using Dijkstra's Algorithm and Machine Learning

A Graph Theory Application

National University of Computer and Emerging Sciences

FAST School of Computing, Karachi Campus

Fall 2025

Course Instructor

Sir Moheez Ur Rahim Khan

Submitted By

Name	Roll Number
Huzaifa Abdul Rehman	23K-0782
Abdul Ahad Munaf	23K-0590
Muhammad Bilal	23K-0026

Abstract

This project implements an innovative approach to route optimization by combining classical graph theory algorithms with machine learning techniques. We developed a system using Dijkstra's shortest path algorithm, enhanced with Gradient Boosting-based travel time predictions, to find optimal routes in real-world road networks. Using OpenStreetMap data for San Francisco, California, USA, we extracted and engineered over 15 features including road types, lane counts, speed limits, and congestion factors to train our ML models. Our results show an average improvement of 18.87% in route efficiency compared to traditional distance-based routing, with best-case improvements reaching 34.60%. This project demonstrates how combining graph algorithms with machine learning can create better navigation systems.

Note: These performance metrics are specific to the San Francisco road network (10,011 nodes, 27,584 edges). Results may vary for different cities based on road network complexity, urban density, infrastructure quality, and traffic patterns.

Keywords: *Dijkstra's Algorithm, Machine Learning, Gradient Boosting, XGBoost, Route Optimization, Graph Theory, OpenStreetMap, Travel Time Prediction, San Francisco*

1. Introduction

1.1 Background and Motivation

Most navigation apps today either minimize distance or estimate travel time based on speed limits. But these simple approaches miss important factors like traffic congestion, road quality, and how different roads actually perform. For example, the shortest route might go through residential areas with many stop signs, taking longer than a slightly longer highway route.

1.2 Our Approach

In this project, we developed a hybrid system that combines Dijkstra's shortest path algorithm with machine learning to predict travel times more accurately. We trained ensemble learning models (Random Forest, Gradient Boosting, and XGBoost) on road network features including congestion factors, road importance levels, and capacity scores. The system finds routes that minimize actual travel time rather than just distance, often discovering faster routes that traditional methods would miss.

1.3 Project Objectives

Our goals for this project were:

- Build a weighted directed graph from real OpenStreetMap data
- Implement Dijkstra's algorithm efficiently for finding shortest paths
- Create a feature engineering pipeline that extracts 15+ attributes from road networks
- Train and compare multiple ML models: Random Forest, Gradient Boosting, and XGBoost
- Show measurable improvements in route optimization compared to traditional methods

2. Methodology

2.1 Data Acquisition and Network Construction

We used the OSMnx Python library to download real street network data from OpenStreetMap for San Francisco, California, USA. The network is represented as a directed graph $G = (V, E)$, where nodes V are intersections (10,011 nodes in our dataset) and edges E are road segments (27,584 edges). The total road network spans 3,126.6 km with speed limits ranging from 16.1 km/h to 104.6 km/h. Each edge has properties like distance, speed limit, and estimated travel time.

2.2 Feature Engineering Pipeline

We extracted 15+ features from the road network data:

- **Intrinsic Geometric Features:** segment length, lane count, posted speed limit (km/h), bridge indicator, tunnel indicator, directional flow constraint
- **Derived Capacity Metrics:** capacity score computed as the product of lane count and speed limit, speed-to-length ratio for segment characterization
- **Traffic Simulation Parameters:** congestion multiplication factors ranging from 1.1 for motorways to 1.6 for residential streets
- **Hierarchical Classification:** importance scoring system assigning values from 5 (major highways) to 1 (residential streets)
- **Geometric Penalties:** turn penalties applied to abbreviated segments (length < 50 meters) to model intersection delays

2.3 Congestion Modeling

Since we don't have real-time traffic data, we created a congestion model based on road types. Different roads have different congestion factors that adjust the travel time:

Road Classification	Congestion Factor (Ci)	Effective Speed Reduction
Motorway/Highway	1.1	10% reduction from posted limit
Primary Arterial	1.3	30% reduction from posted limit
Secondary Road	1.4	40% reduction from posted limit
Tertiary Road	1.5	50% reduction from posted limit
Residential Street	1.6	60% reduction from posted limit

2.4 Machine Learning Models

We trained and compared three ML models for predicting travel times:

- **Random Forest:** Uses 150 decision trees with max depth of 20, combining their predictions for better accuracy
- **Gradient Boosting:** Builds trees sequentially with learning rate 0.1, each tree correcting the errors of previous ones
- **XGBoost:** An optimized version of gradient boosting with regularization to prevent overfitting

We split our data 80/20 for training and testing. We used Mean Absolute Error (MAE) and R^2 score to measure how well each model performs.

3. Implementation Details

3.1 Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path in weighted graphs with non-negative edge weights. With a priority queue, it runs in $O((V + E) \log V)$ time. We implemented three different routing strategies:

Algorithm Variant	Edge Weight Parameter	Optimization Criterion
Distance-Optimal	weight='length'	Minimize geometric distance
Time-Optimal (Baseline)	weight='travel_time'	Minimize time using speed limits
ML-Enhanced	weight='ml_travel_time'	Minimize predicted travel time

3.2 How It Works

Our pipeline follows these steps:

- Step 1:** Download street network data from OpenStreetMap using OSMnx
- Step 2:** Extract and engineer 15+ features from the road data
- Step 3:** Train ML models on travel time data
- Step 4:** Use trained models to predict travel times for all road segments
- Step 5:** Run Dijkstra's algorithm using ML-predicted travel times as edge weights
- Step 6:** Compare results with distance-based and speed-limit-based routing

3.3 Code Structure

We organized our code into three main modules:

Module	Primary Class/Function	Responsibility
ml_models.py	TravelTimePredictionPipeline	ML model training and inference
routing_algorithms.py	RouteOptimizer	Dijkstra's algorithm implementation
shortest_path_routing.ipynb	Main Pipeline	End-to-end workflow orchestration

3.4 Demo Route and Visualization

For demonstration purposes, we tested our routing system on a real-world route from Golden Gate Park to San Francisco International Airport (SFO). Address geocoding was performed using the Geopy library, which converts human-readable addresses to geographic coordinates (latitude/longitude). The results are visualized in an interactive HTML map (route_comparison.html) using the Folium library, allowing users to compare the three routing strategies side-by-side.

4. Results

4.1 ML Model Performance

Here's how our three models performed on the test data:

Model Architecture	MAE (seconds)	R ² Score	Prediction Accuracy
Random Forest	1.60	0.901	90.1%
Gradient Boosting (Best)	1.54	0.904	90.4%
XGBoost	1.55	0.903	90.3%

Gradient Boosting performed best with the lowest error (1.54 seconds MAE) and highest R² score (0.904), meaning it explains 90.4% of the variation in travel times. All three models achieved over 90% accuracy on the San Francisco dataset, demonstrating the effectiveness of our feature engineering approach.

4.2 Route Optimization Performance

We tested 10 random origin-destination pairs and got these results:

Performance Metric	Value	Interpretation
Mean Improvement	18.87%	Average efficiency gain over baseline
Maximum Improvement	34.60%	Best-case scenario performance

Median Improvement	17.76%	Typical case performance
Standard Deviation	12.79%	Performance consistency measure
Average Time Saved	100.3 sec	Per route time savings

4.3 Key Findings

What we learned from our experiments:

- Gradient Boosting is the best model with 1.54 seconds MAE and 90.4% R² score on the San Francisco dataset
- ML-optimized routes consistently outperformed distance-based routing in all tests
- Longer routes (10+ km) showed bigger improvements, meaning ML routing helps more for complex journeys
- Segment length and congestion factors are the most important features, contributing over 60% of the model's predictions
- We created interactive maps using Folium to visualize how the three routing strategies choose different paths

5. Conclusion and Future Work

This project successfully combines classical graph theory with machine learning for better route optimization. By using Gradient Boosting to predict travel times and feeding those predictions into Dijkstra's algorithm, we achieved an average improvement of 18.87% over traditional distance-based routing on the San Francisco road network, with best cases reaching 34.60%. These results demonstrate the significant potential of ML-enhanced routing in complex urban environments.

What we accomplished: (1) implemented Dijkstra's algorithm with three different routing strategies, (2) built a feature engineering pipeline extracting 15+ road attributes, (3) compared three ML models and found Gradient Boosting works best for San Francisco (90.4% accuracy), (4) showed consistent improvements averaging 18.87% across different routes, and (5) created interactive maps to visualize the results.

5.1 Limitations and Future Work

Our project has some limitations. The congestion factors we used are simulated, not real-time data, so accuracy might vary in unusual traffic conditions. The model also doesn't account for time-of-day variations like rush hour. Additionally, our results are specific to the San Francisco road network—performance metrics (improvement percentages, best model selection) may vary for different cities based on road network topology, urban density, and infrastructure characteristics.

Ideas for future improvements:

- Add real-time traffic using APIs like Google Maps or TomTom
- Include time-of-day features for rush hour predictions
- Try Graph Neural Networks (GNNs) for better spatial pattern recognition
- Add multi-objective optimization for fuel efficiency and emissions
- Test on different city networks to validate cross-location performance
- Compare results across multiple cities (e.g., New York, Los Angeles, Chicago) to understand how urban topology affects optimization gains